SpMV–G1 / CSR SpMV: correctness and memory model

Weight: 8% of the course grade | Due date: to be announced

Objective and fixed contract

Show what CSR SpMV computes, prove the supplied kernel is correct, and predict memory/cache behavior before profiling. A = R×C CSR, y = A x; R=65,536, C=4R, eight nonzeros/row, 64-bit values/x/y, 32-bit column indices, 64-bit row offsets. Use the supplied C++ source, without modifying the computational kernel for the baseline.

Required tasks

Task	Evidence students must produce	Marks
1. Functional contract	Define A, x, y, NNZ, input/output invariants; name one SpMV invocation.	1
2. Golden correctness	Derive the toy CSR arrays and output below; run self-test and both layouts; compare checksums.	2
3. Memory model	Compute bytes for row_ptr, col_idx, values, x, y, expected; calculate logical x demand per call; do not label it DRAM traffic.	2
4. Cache mapping	For trace [0,1,8,9,0,8,1,9] ×3, base 0x1000, 4 B/element, 16 B/line, 32 B direct mapped: derive line ID, set, offset, 24 H/M decisions; contrast equal-capacity 2-line fully associative.	2
5. Provenance	Record compile flags, full commands, source SHA256 and PASS output.	1

Independent toy example for Task 2

A has 3 rows × 4 columns: row 0 nonzeros (0,2), (3,1); row 1 (1,−1), (2,3); row 2 (0,4). Let x=[1,2,3,4]. Compute row_ptr, col_idx, values, y by hand. The built-in --selftest is a supplementary check; an explanation is required.

Assumptions and boundaries

The toy cache uses 4-byte x elements and is independent of the real benchmark (8-byte doubles). Distinguish memory-block ID from resident cache slots. The array expected is an oracle, not kernel payload.

Start-to-finish commands • execute inside extracted package directory

```
g++ -O3 -g -fno-omit-frame-pointer -std=c++17 -Wall -Wextra \
  SpMV_Guided_Benchmark.cpp -o spmv_bench
./spmv_bench --selftest
./spmv_bench --pattern local --rows 65536 \
  --nnz-per-row 8 --iters 500
./spmv_bench --pattern permuted --rows 65536 \
  --nnz-per-row 8 --iters 500
sha256sum SpMV_Guided_Benchmark.cpp di_common.hpp di_kernel_pmu.hpp
```

Submission (one folder: SpMV_G1_<studentID>)

- report.pdf (maximum 3 pages): toy worked example; six-array memory table with units; 24-reference hit/miss table; why mapping and associativity matter; both checksum values.
- terminal.txt: complete build, self-test and two baseline outputs; source_hashes.txt; any modified source files (if changes were made).
- Include answers in your own words. Correctness evidence is necessary; a printed PASS line alone is insufficient.

Acceptance and scoring

Correct CSR outputs and an independently explained toy calculation are essential for the two correctness marks. Analytical tables must state array element widths, cache parameters and calculation formulas. If the program fails, supply the full error log; fabricated PASS/results receive no credit.

Evaluation notes

This assignment is assessed on mathematical correctness, the CSR representation, memory calculations, and a fully documented execution. Keep the raw commands and outputs so another person can reproduce the result.